

Propositions

Sehyun Kwon

Mathematical Data Science
Hanyang University ERICA

Testing is not proof

A data pipeline contains a function that is claimed to return the largest value in any nonempty list of numbers.

Opening question

How many successful tests would make you *certain* that the function works for every nonempty list of numbers?

10?

10,000?

all possible inputs?

Successful tests give evidence; they do not make failure impossible for every possible list of numbers.

Why discrete mathematics for data science?

Discrete mathematics provides foundational language and reasoning tools for both data science and computer science.

1. How can we state precisely what an algorithm, model, or data pipeline guarantees?
2. How can we reason about finite structures such as graphs, datasets, and programs?

These questions lead to logic, proofs, counting, and graph theory.

Mathematical arguments

Claim, assumption, proof

- ▶ A **claim** is a declarative statement presented as true.
- ▶ An **assumption** is a statement accepted as a starting condition for an argument.
- ▶ A **proof** is a finite sequence of justified statements that establishes a claim from stated assumptions and definitions.

A proof is about a precisely stated claim; it is not a longer list of tests.

Finding the maximum

```
def largest(x):  
    m = 0  
    for value in x:  
        if value > m:  
            m = value  
    return m
```

Claim

For every nonempty list of real numbers, `largest(x)` returns its largest entry.

Does the claim survive these tests?

`[2, 9, 4]`

`[0.3, 0.8, 0.1]`

`[-8, -3, -5]`

A counterexample

For the input list $[-8, -3, -5]$:

the function returns 0,

but the largest entry is -3 .

Counterexample

A **counterexample** to a claim is one allowed case in which the claim is false.

One counterexample defeats a claim about every allowed case; many passing tests do not prove it.

Notation

Let n be a **positive integer**: one of $1, 2, 3, \dots$

Let $x = (x_1, x_2, \dots, x_n)$ be an **ordered list** with n entries.

- ▶ x_i means the entry in position i .
- ▶ $\mathbb{R}$ denotes the set of **real numbers**.
- ▶ $x_i \in \mathbb{R}$ means “ x_i is an element of $\mathbb{R}$.”
- ▶ $\max\{x_1, \dots, x_n\}$ means the largest value among the listed entries.

Corrected algorithm

```
def largest(x):  
    m = x[0]  
    for value in x[1:]:  
        if value > m:  
            m = value  
    return m
```

Here Python uses $x[0]$ for the first entry and $x[1:]$ for the remaining entries; our mathematical list uses x_1 for the first entry.

Required output

For an input $x = (x_1, \dots, x_n)$, where n is positive and each $x_i \in \mathbb{R}$, the function must return

$$\max\{x_1, \dots, x_n\}.$$

Why does the algorithm work?

Let i be the number of entries inspected so far, and let m be the value currently stored by the algorithm.

After the first entry is inspected,

$$m = x_1 = \max\{x_1\}.$$

Suppose $1 \leq i < n$ and $m = \max\{x_1, \dots, x_i\}$. After x_{i+1} is inspected, the update gives

$$m = \max\{x_1, \dots, x_i, x_{i+1}\}.$$

After all n entries have been inspected,

$$m = \max\{x_1, \dots, x_n\}.$$

Therefore, the algorithm returns the maximum value.

Propositions

Proposition

A **proposition** is a declarative statement that has exactly one truth value: **true** or **false**.

We abbreviate the truth values by

T for true, **F** for false.

A proposition need not be easy to verify.

Example

“The repaired function returns the largest entry of every nonempty list of real numbers” is a proposition even before the claim has been proved.

Which sentences are propositions?

1. $10^2 = 100$.
2. $10^2 = 90$.
3. $x > 10$.
4. This model is good.
5. Retrain the model.

T **F** ? ? ?

Items 3–5 are not propositions as written: a value, criterion, or truth-valued assertion is missing.

Open sentences

An **open sentence** contains a variable whose allowed value has not been fixed or quantified.

$$x > 10$$

The open sentence $x > 10$ becomes a proposition after assigning a value to x :

$$\text{when } x = 12, \quad 12 > 10 \text{ is } \mathbf{T}.$$

A **quantifier** specifies how a variable ranges, using phrases such as “for every value” or “for at least one value.” Formal quantifier notation begins in Lecture 02.

Before assigning or quantifying x , the expression $x > 10$ has no single truth value.

Measurable claims

“This classifier is good” has no agreed truth value.

In this example:

- ▶ a **classifier** is a rule that assigns a label to an input;
- ▶ **test accuracy** is the fraction of test examples labeled correctly.

The following sentence is a proposition:

Measurable claim

“The classifier’s accuracy on the fixed test set is at least 0.90.”

Propositional variables

A **propositional variable** is a symbol that stands for a proposition.
Let

P = “the model has been trained,”

Q = “the model has been evaluated.”

Each of P and Q has truth value **T** or **F**.

Logical connectives

A **logical connective** combines or modifies propositions to form a new proposition.

For propositions P and Q , the basic connectives are:

name	symbol	read as
negation	$\neg P$	not P
conjunction	$P \wedge Q$	P and Q
disjunction	$P \vee Q$	P or Q

A proposition formed using connectives is a **compound proposition**.

Negation

Negation

$\neg P$, read “not P ,” is true exactly when P is false.

If P is “the model has been trained,” then

$\neg P$ is “the model has not been trained.”

P	$\neg P$
T	F
F	T

Conjunction

Conjunction

$P \wedge Q$, read “ P and Q ,” is true exactly when both P and Q are true.

The model has been trained *and* evaluated:

$$P \wedge Q.$$

For which truth values is the conjunction true?

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

Disjunction

Disjunction

$P \vee Q$, read “ P or Q ,” is true exactly when at least one of P and Q is true.

Mathematical “or” is **inclusive**: it includes the case where both are true.

P	Q	$P \vee Q$
T	T	T
T	F	T
F	T	T
F	F	F

Truth tables

A **truth assignment** chooses **T** or **F** for each propositional variable.

A **truth table** lists every truth assignment and the resulting truth value.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$
T	T	F	T	T
T	F	F	F	T
F	T	T	F	T
F	F	T	F	F

Two variables produce $2^2 = 2 \times 2 = 4$ assignments because each variable has two possible truth values.

Operator precedence

When several logical connectives occur in one proposition, their order matters.

Parentheses specify how the proposition is grouped.

When parentheses are omitted, we use the following **precedence convention**:

1. negation $\neg$
2. conjunction $\wedge$
3. disjunction $\vee$

Parentheses override this convention.

Exercise: translation

Consider one observation in the dataset. Define

O = “the value is an outlier,”

L = “the label is missing.”

Write in symbols

Write a symbolic form for each sentence.

1. The observation is an outlier and its label is not missing.
2. The observation is an outlier or its label is missing.
3. Neither condition holds.

Solution

Recall:

O : outlier, L : label missing.

- | | |
|-----------------------------------|----------------------------|
| 1. outlier and label not missing: | $O \wedge (\neg L)$ |
| 2. outlier or label missing: | $O \vee L$ |
| 3. neither condition holds: | $(\neg O) \wedge (\neg L)$ |

Break a statement into small propositions, then combine them with logical connectives.

Common mistakes

1. **Unbound variable:** treating $x > 10$ as already true or false.
2. **Vague criterion:** calling a model “good” without a metric and threshold.
3. **Misreading $\vee$:** assuming that $P \vee Q$ is false when both P and Q are true.
4. **Hidden grouping:** writing a long formula without parentheses or a stated precedence rule.

Debugging habit

When a formula feels ambiguous, return to the plain-language propositions and their definitions.

Summary

- ▶ Distinguish testing evidence from a mathematical guarantee.
- ▶ Decide whether a sentence is a proposition.
- ▶ Represent propositions with propositional variables.
- ▶ Read and evaluate $\neg$, $\wedge$, and inclusive $\vee$.
- ▶ Build a truth table from all truth assignments.

Next: implication, equivalence, and the exact logic of “if” and “only if.”